AI ENGINEER / AGENTS & AUTOMATION

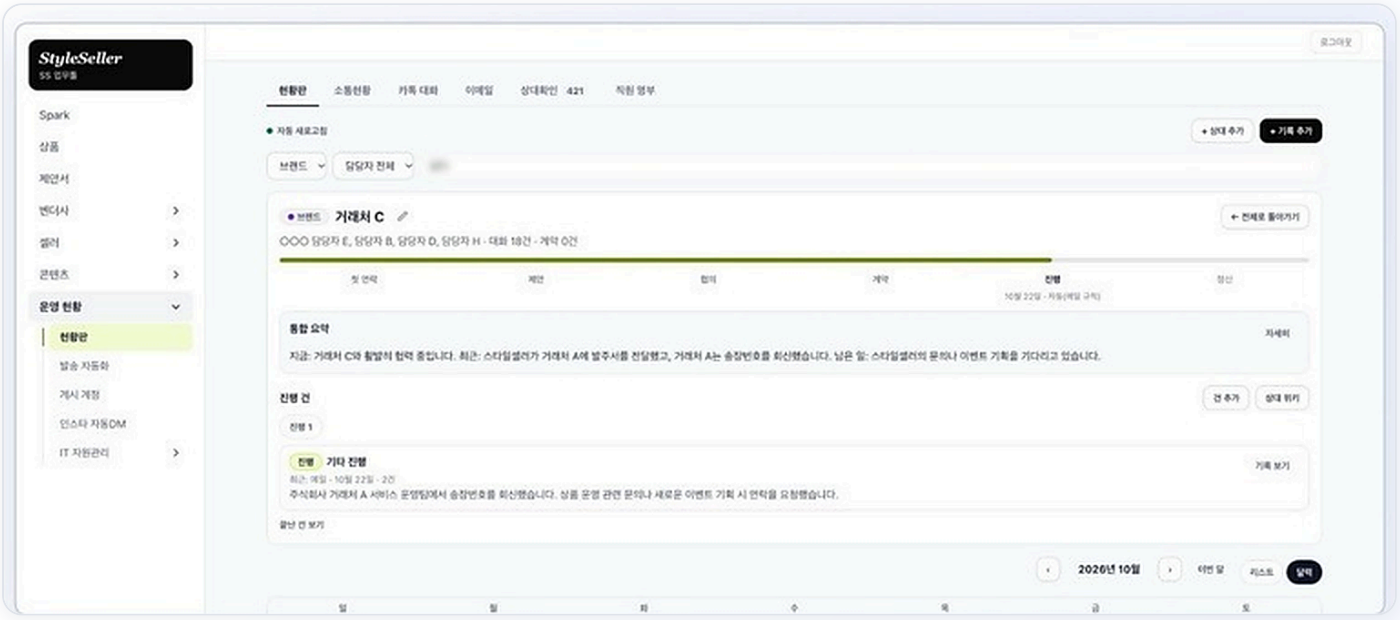
Portfolio

“현업의 반복 업무를,
매일 돌아가는 AI 시스템으로”

목진희

AI 엔지니어 · 설계부터 배포·운영까지

StyleSeller에서 카톡·메일 기록 정리, 거래처 CS, 사내 업무툴, 에이전트 플랫폼을 만들고 직접 배포해 운영합니다. 직원 PC에서 실제로 돌려 본 뒤에야 끝났다고 말합니다.



실제 운영 중인 현황판: 상대 한 곳의 단계 막대, 통합 요약, 달력 (이름은 가렸습니다)

01
현황판
운영 원장
카톡·메일·슬랙 기록을
상대 수백 곳 단위로 한 화면

02
콘텐츠
오토파일럿
릴스 요청부터 승인까지
게시는 승인 뒤 기본 수동

03
벤더 CS
에이전트
태그된 질문에만, 인계 없이 답변
금액·브랜드는 코드로 대조

04
에이전트
플랫폼·Spark
CEO 배정, Jev 스킬 추천
도구 게이트, 비용 상한

05
지식 허브
메일 에이전트
LightRAG 워크스페이스 13개
근거 있는 초안만

06
사내 업무툴
가격은 코드로 계산
맞춤카톡 발송 자동화

07
직원 PC
배포·운영
11대 한 줄 설치
기기별 키

목진희
Jinheemok815@gmail.com

STYLESELLER
https://jin-tonix.github.io/

목진희

AI·자동화 엔지니어 · 설계부터 배포·운영까지

이력서

jinheemok815@gmail.com · 010.9030.2888
jin-tonix.github.io
서울시 강남구 역삼동

요약

직원이 손으로 반복하던 일을 AI 에이전트와 자동화로 바꾸고 직접 운영합니다. 2025년 11월부터 StyleSeller에서 거래처 CS·메일 답장·사내 데이터 질의 에이전트, 사내 업무 웹, 직원 PC 배포까지 설계·구현해 운영하고 있습니다. 법학 전공 뒤 어린이집 운영실장과 영어 강사로 8년 일하고 개발로 전환했습니다. 코드는 AI 코딩 도구(Claude Code)에 많이 맡기고, 범위 설정과 결과 확인은 직접 합니다.

경력

스타일셀러 (StyleSeller) · AI·자동화 PL

2025.11 ~ 재직 중 (약 1년)

- LLM 에이전트 설계·개발·운영: 거래처 CS 에이전트, 메일 답장 에이전트, 사내 데이터 질의 비서
- RAG·지식 검색: LightRAG(Neo4j + pgvector) 지식 허브, 초안 주장 단위 근거 검증
- 업무 자동화: 카카오톡·메일 수집, 공급사 카탈로그 상품 등록, 가격 계산·파트너 매칭 자동화
- LLM 평가·개선: 스킬 라우터 정답률 90.15%(132건), 정답표 오류·가짜 스킬 정정 후 재측정
- 배포·운영: GCP VM·Docker·NGINX, 직원 PC 11대 한 줄 설치·자동 갱신 배포, DB 장애 당일 복구와 4개 자체 호스팅 이전

위리드 (We:Review) · AI 개발 인턴

2025.08 ~ 2025.10 (3개월)

- 영수증 처리 자동화: PaddleOCR·DBSCAN·Qwen2.5-7B로 구조화 데이터 추출, FastAPI 서비스화
- 검수 정확도 70%→95%, 검수 시간 85% 단축 · 실서비스 배포까지 참여

정상어학원 · 파츠앤팬스 · 몬테키즈어학원 · 영어강사

2018.08 ~ 2024.02

5~15세 대상 영어 몰입 수업 설계·교안 제작, 학생별 학습 계획 운영, 정기 학부모 상담.

전원어린이집·방과후학교 · 운영실장

2015.03 ~ 2018.02 (3년)

교사 채용, 연간 커리큘럼·행사 기획, 학부모·운영위원회 소통, 원장 대행 운영 관리.

학력

청주대학교 · 법학과 2008.03~2012.02

학점은행제 · 경영학과 2021.02~2023.06

교육

하이미디어 · 풀스택(960h) 2024.04~10

하이미디어 · AI 심화 2024.10~12

한국IT비즈니스진흥협회 · 생성형 AI 2025.05~07

스킬

AI·LLM	Gemini, LiteLLM, LightRAG, RAG, Tool calling, MCP, 평가셋 설계·측정
백엔드	Python, FastAPI, TypeScript, Node.js, PostgreSQL
인프라	GCP, Docker, NGINX, Vercel, GitHub Actions, n8n

한눈에 보기

StyleSeller에서 만들고 운영하는 시스템 7개와 후속 개선 1건입니다. 사례마다 어떻게 만들고 왜 그렇게 정했는지 설명합니다. 수치는 각 사례의 기준일로 적었습니다.

현황판 · 운영 원장

카톡·메일·슬랙 기록을 상대별 달력에 모으고, 수백 곳의 거래 단계를 한 화면에 보여 줍니다

콘텐츠 오토파일럿

릴스 요청부터 렌더·승인까지 실무자 화면에서 하고, 게시는 승인 뒤 기본 수동입니다

벤더 CS 에이전트

거래처 단톡방에서 태그된 질문에만, 인계 없이 답하고, 금액·브랜드는 코드로 근거와 대조합니다

에이전트 플랫폼 · Spark

CEO 작업지시서에서 시작해 디스패처, 작업자, 공용 스킬(Jev 추천), 도구 게이트까지 서버가 통제합니다

Jev 스킬 라우터 고도화 (Spark 후속)

Hermes 에이전트의 스킬 선택(Jev)을 정확하게 만들고 측정 오류를 바로잡음. Jev로 먼저 거르면 비용이 약 1/10로 줄 것으로 추정

LightRAG 지식 허브 · 메일 에이전트

에이전트 4곳이 같은 지식 그래프에서 근거를 찾고, 메일은 근거 있는 초안만 담당자에게 갑니다

사내 업무툴

가격은 시트 수식을 코드로 옮겨 계산하고, LLM은 고칠 칸만 돌려줍니다. 맞춤카톡 발송을 자동화했습니다

직원 PC 배포 · 운영

카톡·인스타 작업을 직원 PC 11대에 한 줄 설치로 배포했습니다

현황판

현황판 · 운영 원장

상대 한 곳을 고르면 카톡·메일·주문서가 한 달력에, 수백 곳의 거래 단계가 한 화면에 보입니다

기간	2026.10.19~10.23 구현(5일), 운영 중
역할	설계: 원장(기록을 한 번씩만 쌓는 표) 스키마, 상대 연결 규칙 / 구현: 인입 API, 메일 수집기, 화면 탭 6개 / 검증: DB 시험, 운영 화면. AI 코딩 도구를 썼고, 범위는 제가 정하고 결과는 시험으로 확인했습니다
저장소	현황판 관련 커밋 사내 저장소 2곳 약 150

배경

거래 상대와 나눈 이야기는 카톡방, 주문 메일함, 슬랙에 나뉘어 있고 엑셀 주문서는 메일 첨부로 따로 옵니다. 현황판은 이 기록을 하루 단위로 요약해 상대별 달력에 날짜순으로 올립니다. 브랜드·벤더사·셀러 중 한 곳을 고르면 그 상대와 오간 기록이 한 화면에 모입니다. 소통현황 탭은 수백 곳이 첫 연락부터 정산까지 어느 단계인지 보여 줍니다. 규칙으로 못 이은 기록이나 단계가 애매한 건은 직원이 정합니다.

구조와 규모

1. 기술

React 19.2·Tailwind 4·Vitest 4, Python 3.12·Pydantic v2, Postgres RLS(행마다 볼 수 있는 사람을 제한하는 DB 규칙), Gemini Flash 계열(LiteLLM 경유)

2. 규모 (항목별 기준일 표기)

- 소통현황 단계별 비중: 첫 연락 13.1%, 제안 33.8%, 협의 2.6%, 계약 4.5%, 진행 46.0%, 정산 0% (2026-10-24)
- 평일 하루 기록 100~200건 안팎, 상대확인 대기 수백 건 (2026-10-24)

3. 어떻게 동작하나

① 카톡은 하루 3회(09:15·13:00·17:55), 주문 메일함은 5분마다, 슬랙은 범용 커넥터로 들어옵니다. 같은 기록이 몇 번 와도 원장에는 한 번만 남습니다.

② 뉴스레터는 LLM 전에 규칙으로 빼고, 주문서 표에서는 수령인 같은 열을 찾아 구매자 이름을 가립니다. 요약이 근거로 든 문구가 원문에 없으면 그 요약은 버립니다.

③ 상대 연결에는 LLM을 쓰지 않고 규칙만 씁니다. 메일 스레드, 사업자번호, 카톡방, 이메일, 전화, 도메인, 거래처 코드, 별칭 순으로 보고, 키가 엇갈리면 미분류에 둡니다.

④ 진행 건은 카톡 계약 신호와 메일 신호를 상대·대상·회차 키로 묶습니다. 단계는 규칙 근거가 분명하면 자동 반영하고, 나머지는 '확인 대기'로 올려 직원이 정합니다.

⑤ 매일 18:05 상대 최대 500여 곳의 요약을 3~6줄로 다시 씁니다. 하루 2,000원을 넘거나 비용 장부를 못 읽으면 그날 배치는 멈춥니다.

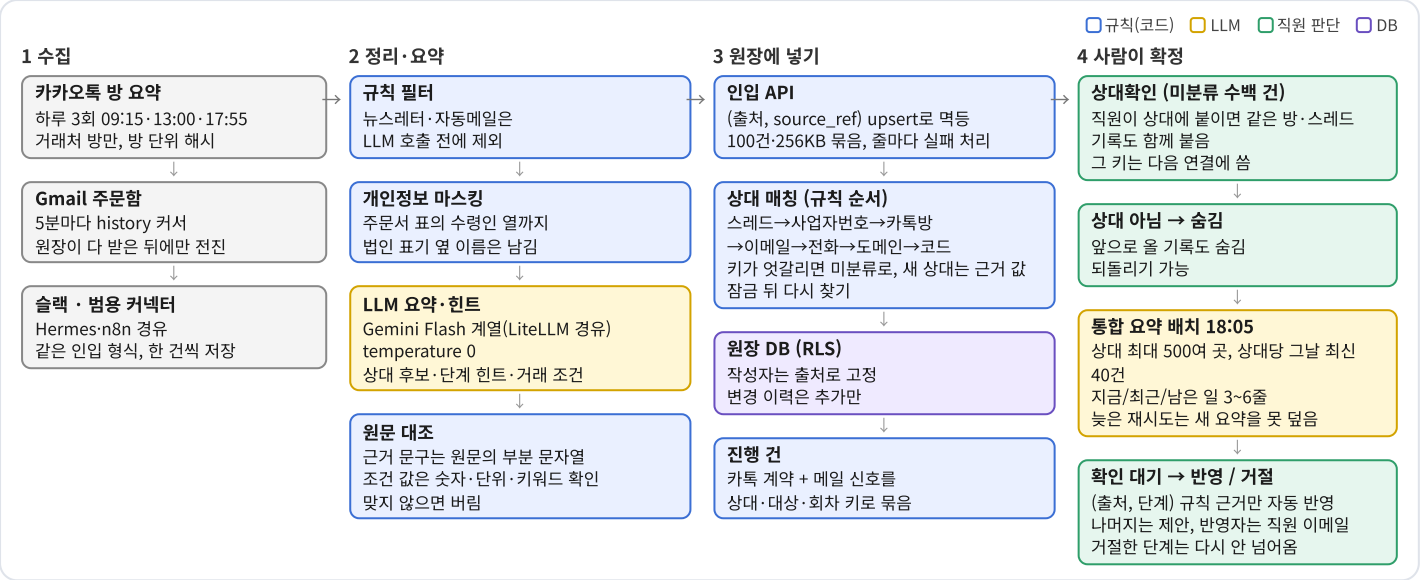
사고와 수정

1. **요약 107건 전부 거절(2026-10-23).** 요약을 저장하는 DB 함수에 앱 역할의 실행 권한이 없어서 그날 요약 107건이 저장 단계에서 모두 db_error로 떨어졌습니다. 그 함수 하나에만 실행 권한을 주는 GRANT 한 줄로 풀었습니다.

2. **리허설에서 '진행' 제안 과다(2026-10-23).** 규칙이 '배송·재고·매출' 같은 단어만 나와도 거래 신호로 읽었습니다. 실제 거래 신호 단어만 남기고, '판매중단'·'발주문의' 같은 부정 표현은 먼저 지운 뒤 맞추게 바꿨습니다.

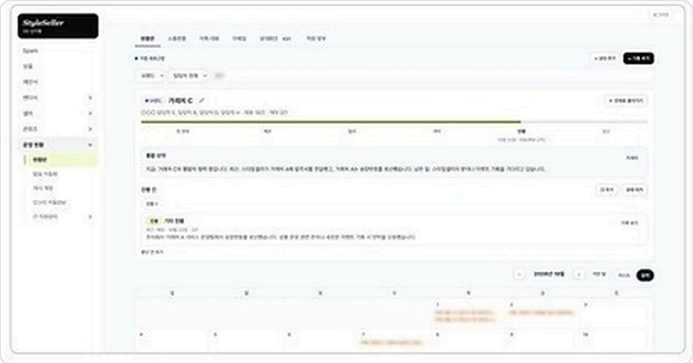
3. **공용 챗봇이 담당자로 표시(2026-10-23).** 상대 64곳의 담당자 칸에 공용 챗봇 이름이 들어갔습니다. 카톡방에서 말을 많이 한 상위 5명을 담당자로 잡았는데, 그 안에 챗봇이 있었습니다. 지금은 회사 메일 계정(@sample-corp.co.kr)만 담당자로 인정하고, 없으면 거래처 프로필의 담당자를 씁니다.

Architecture

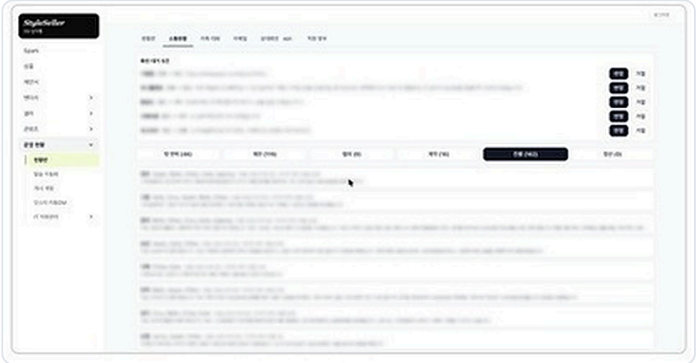


카톡·메일·슬랙 → 서버(마스킹, LLM 추출, 원문 대조) → 멱등 인입(여러 번 와도 한 번만 저장) → 업무툴 원장(RLS, 규칙 매칭) → 직원 확인

화면



화면 A: 상대별 달력. 브랜드·벤더사·셀러 한 곳을 고르면 카톡 요약, 메일, 엑셀 주문서가 날짜순 달력 하나에 놓입니다. 위에는 단계 막대와 통합 요약(지금/최근/남은 일), 진행 건, 상대 위키가 붙습니다.



화면 B: 소통현황. 단계별 상대 비중(첫 연락 13%~진행 46%)과 '확인 대기' 제안이 보이고, 직원이 반영이나 거절을 누릅니다.

기술 하이라이트

1. 같은 기록이 다시 와도 화면은 그대로입니다. 수집기는 같은 기록을 몇 번이고 다시 보냅니다. (출처, source_ref)가 같으면 갱신하는 upsert 한 문장으로 저장합니다. 직원이 고친 행과 내용이 같은 행은 건드리지 않습니다. 내용이 같으면 트리거도 순번과 시각을 올리지 않아서, 재전송 뒤에도 화면이 그대로입니다.

2. 작성자 값은 호출자가 보낸 것을 쓰지 않습니다. 인입 함수는 호출자가 보낸 작성자 값을 버리고 'ingest:출처'로 다시 만듭니다. manual 출처는 직원 전용이라 거절합니다. '직원이 고침' 표시는 앱이 아니라 트리거가 찍고, 숨긴 기록은 UPDATE로 되살릴 수 없습니다. 앱 역할에는 함수 실행 권한만 줍니다.

3. 키가 엇갈리면 잇지 않습니다. 연결 순서는 스레드, 사업자번호, 카톡방, 이메일, 전화, 도메인, 거래처 코드, 별칭으로 고정했습니다. 사업자번호나 전화가 다른 근거와 다른 상대를 가리키면 충돌로 표시하고 미분류에 둡니다. gmail·naver 같은 공용 도메인 13개는 근거로 쓰지 않습니다.

4. 동시 수집이 상대를 두 번 만들지 않습니다. 새 상대를 만들 때만 근거 값마다 advisory lock(트랜잭션 간 이름표 잠금)을 정렬 순서로 잡고 다시 찾고, 그 사이 다른 트랜잭션이 만든 상대가 있으면 그것을 씁니다. 직원의 단계 반영도 수집과 같은 잠금 순서(상대 → 제안)를 따라서 교착이 생기지 않습니다.

5. LLM 출력은 원문과 맞춰 봅니다. 단계 근거 문구는 마스킹된 본문의 부분 문자열일 때만 남깁니다. 거래 조건은 더 엄격합니다. 값에 든 숫자가 하나하나 인용문에 있어야 하고, 조건 종류별 키워드와 단위도 맞아야 합니다. 회사명은 근거 줄 안에 정규화한 이름이 있을 때만 씁니다.

6. 주문서 표는 열 단위로 가립니다. 주문서는 대개 표로 옵니다. '수령인·주문자' 같은 머리 열을 찾고, 칸 수가 머리 줄과 같은 줄에서만 그 열의 값을 가리니 상품명과 수량은 요약에 남습니다. '(주)' 같은 법인 표기 옆 이름은 회사명으로 보고 가리지 않습니다.

7. 비용은 부르기 전에 예약합니다. 잔액 검사와 예약이 SQL 한 문장이라 동시 호출 둘이 함께 한도를 넘지 못합니다. 예약액은 최대 입출력 기준 추정치와 최소 예약액 중 큰 쪽입니다. 정산이 실패하면 그 뒤 호출을 멈춥니다. 늦게 온 재시도 요약은 날짜 비교에 걸려 새 요약을 덮지 못합니다.

8. 메일 커서는 원장이 다 받은 뒤에 옮깁니다. 메일은 로컬 큐에 먼저 넣고 원장으로 보냅니다. Gmail history 커서는 그 회차 전달이 모두 성공해야 옮깁니다. 커서가 만료되면 마지막 성공 시각부터 목록으로 다시 채웁니다. 두 번 실패한 메일은 격리하고 경보를 냅니다.

상태 · 판단 · 한계

- 운영 중입니다. 6개 탭, 카톡·메일·슬랙 인입, 18:05 요약 배치가 돌고 있습니다(2026-10-24 운영 화면 확인).
- 이름·전화 정규화 정답 파일을 Python, TS, SQL이 함께 씁니다. CI는 내용 검증 시험을 PGlite 위에서 실제 앱 역할로 실행하고, 화면 시험은 CI 밖에서 실행합니다.
- 한계: '확인 대기' 제안을 직원이 반영한 비율, 즉 제안 정확도는 재지 않았습니다. 서버는 근거 문구를 다시 검사하지 않고 인입 쪽 검사를 믿습니다.
- 다시 한다면: 새 DB 함수마다 앱 역할로 불러 보는 시험을 함께 만들겠습니다. 이 시험이 있었다면 10-23 요약 107건 거절은 배포 전에 잡혔을 문제입니다.

내가 정한 것

- 상대 연결은 LLM 대신 규칙 순서로 정했습니다. 키가 엇갈리면 자동으로 잇지 않고 직원에게 넘깁니다.
- 단계 자동 반영은 (출처, 단계) 규칙 표로 하나씩 켭니다. 처음 켠 것은 카톡 출고·발주·공구 → '진행' 하나입니다. LLM 자기 확신도는 보정이 안 돼 쓰지 않습니다.
- 직원이 거절한 단계는 다시 제안하지 않아서 확인 대기열에는 아직 아무도 판단하지 않은 제안만 남습니다.

콘텐츠

콘텐츠 오토파일럿

StyleSeller · 사내 비공개 저장소

릴스 요청부터 9:16 MP4 렌더와 승인까지 실무자 화면에서 끝냅니다. SNS 실게시는 시뮬레이션입니다

기간	2026.10.03~(설계서 승인 후)
역할	설계: 단계 상태기계와 게시 기본값. 구현: 오케스트레이터, 게시 관문과 DB 슬롯, QA 관문. 검증: 운영 canary(한 건만 먼저 돌려 보는 시험)와 재시작 시나리오 테스트. AI 코딩 도구를 썼고, 범위는 제가 정하고 결과는 직접 확인했습니다
저장소	오케스트레이터, 콘텐츠 에이전트, 영상 렌더러, 업무툴(콘텐츠 화면)

배경

실무자가 업무툴 콘텐츠 화면에서 릴스를 요청하고, 미리보기를 본 뒤 승인하고, 배포 일정을 잡습니다. 생성에는 돈이, 게시에는 외부 노출이 걸리니 기본값을 모두 '하지 않는 쪽'에 뒀습니다. 게시는 수동, 반복 자동화는 dry-run(실행하는 흉내만 냄), 유료 생성은 꺼짐입니다. 단계 진행은 코드가 맡습니다.

구조와 규모

1. 기술

Paperclip 하위 이슈, 오케스트레이터(LLM 없는 상태기계), Hermes와 Gemini Flash 계열(LiteLLM 경유), image-service, TTS, 영상 렌더러, HyperFrames(HTML로 영상을 렌더하는 도구), ffprobe, Google Drive, Blotato(SNS 예약 게시 서비스), Supabase(PostgreSQL)

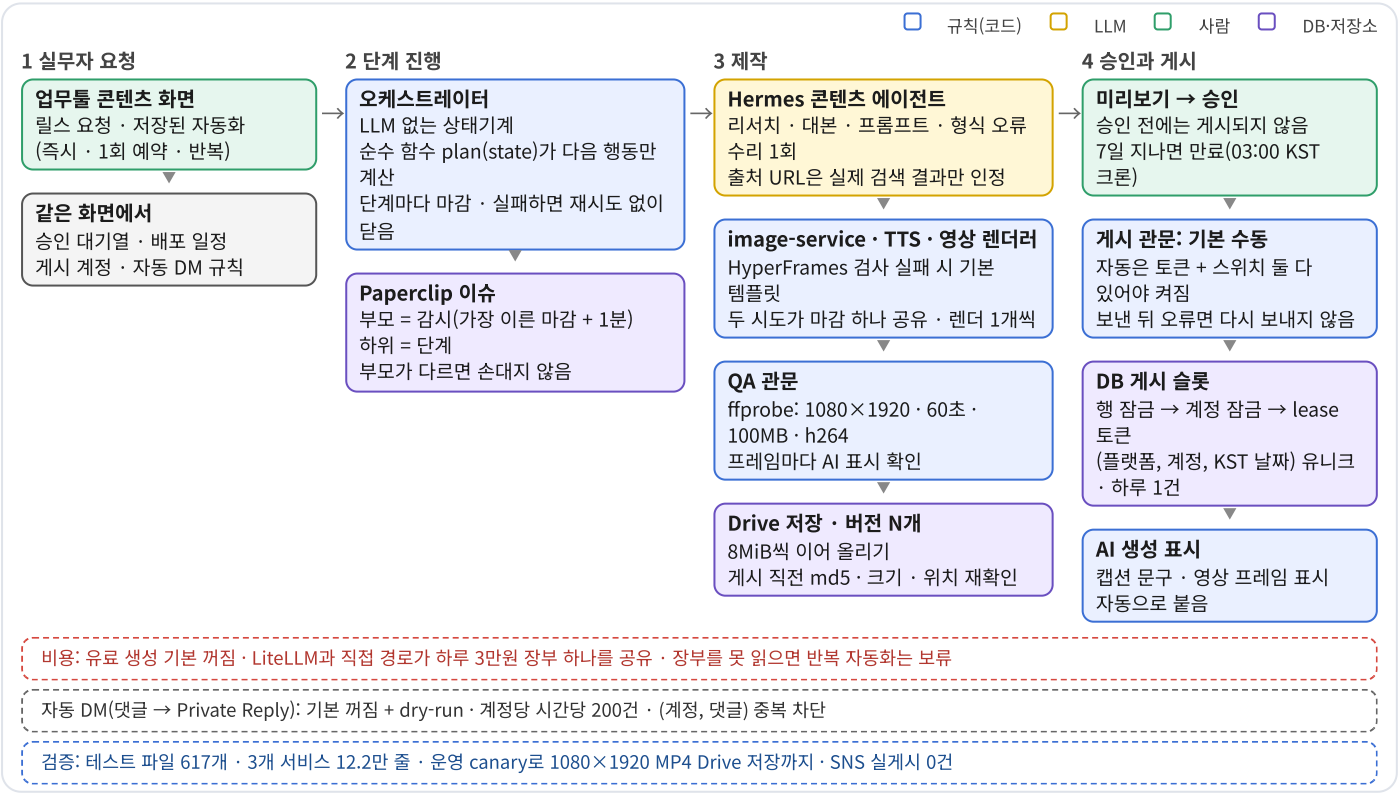
2. 규모 (항목별 기준일 표기)

- 서비스 3개 합 12.2만 줄, 테스트 파일 617개 (오케스트레이터 135, Hermes 에이전트 396, 영상 렌더러 86)
- 운영 canary로 1080×1920 MP4의 Drive 저장까지 검증 (2026-10)
- SNS 실게시 0건. 계정당 하루 게시 1건은 DB가 강제, 게시 관문 상한은 하루 10건 (2026-10-24)

3. 어떻게 동작하나

① 실무자가 릴스를 요청하거나, 저장된 자동화(즉시, 1회 예약, 반복)를 돌립니다.
② 오케스트레이터의 순수 함수가 다음 행동만 계산하면 실행기가 이를 Paperclip 하위 이슈에 반영해 단계를 넘깁니다.
③ Hermes 에이전트가 리서치, 대본, 프롬프트를 쓰고 영상 렌더러가 HyperFrames 템플릿으로 9:16 MP4를 렌더합니다.
④ QA 관문을 통과한 버전은 체크섬과 함께 Drive에 저장됩니다. 사람이 미리보기로 승인하고, 7일 지난 대기는 만료됩니다.
⑤ 게시는 기본이 수동이라 MP4를 받고 캡션을 복사해 '게시 완료'를 누릅니다. 'AI 생성 콘텐츠' 문구와 프레임 표시는 자동으로 붙습니다.

Architecture



요청 → 상태기계(Paperclip 이슈) → Hermes 대본 → 렌더·QA → 사람 승인 → 게시 관문(기본 수동, DB 슬롯)

기본값으로 막은 것

1. 게시. 기본은 manual입니다. 자동 게시는 토큰과 스위치가 모두 있어야 켜집니다. (플랫폼, 계정, KST 날짜) 부분 유니크 인덱스로 계정당 하루 1건만 허용하고 두 번째부터 거절합니다. 기존 중복이 있으면 마이그레이션이 먼저 멈춥니다.

2. 유료 생성과 반복 자동화. 유료 백엔드는 기본 False입니다. 반복 자동화는 dry-run이 기본이고, 값이 정확히 "false"일 때만 풀립니다. 시작 전 회사 하루 3만원 장부를 읽고, 못 읽으면 보류합니다. 상한은 env로 못 올리게 min()으로 묶었습니다.

3. 인스타 자동 DM. 켜기 스위치는 꺼짐, dry-run은 켜짐이 기본입니다. 계정당 시간당 200건이고, 잘못된 설정값은 안전한 쪽으로 읽습니다. (계정, 댓글) 유니크 인덱스가 한 댓글에 DM이 두 번 가는 것을 막습니다.

기술 하이라이트

1. 다음 행동은 순수 함수가 정합니다. plan(state)가 행동 목록만 돌려주고, 실행기가 반영합니다. 단계마다 마감이 있고, 부모 이슈는 가장 이른 마감 + 1분에 깨어 감시합니다. 실패하면 재시도 없이 하위 이슈를 취소하고 닫습니다. 부모가 다른 하위 이슈는 건드리지 않으니 재실행해도 파이프라인이 새로 생기지 않습니다.

2. 형식 오류는 한 번만 고치게 합니다. 수리 대상은 JSON 형식 오류뿐입니다. 이전 출력은 빼고 오류 목록(최대 8개, 각 120자)만 보내며 호출은 최대 2회입니다. 수리 뒤에도 틀리면 세 번째 호출 없이 실패로 끝나는 것을 테스트로 고정했습니다.

3. 출처는 실제 검색 결과만 인정합니다. 인용 URL은 웹 도구 결과나 입력에 있어야 통과하고 모델이 직접 쓴 근거 필드는 버립니다. 검색했다고 주장만 하면 web_research_unverified로 실패시킵니다. 상품 가격 같은 사실은 아직 프롬프트 규칙으로만 막고 있습니다.

4. 결과를 모르는 게시는 다시 보내지 않습니다. POST 직전에 시작 표식을 쓰고 보낸 뒤 오류가 나면 실패 대신 publish_pending으로 둡니다. 재시작 때 이미 게시됐으면 already_published로 닫습니다. Blotato는 제출 ID를 바로 저장해, 다시 깨어나도 상태만 조회합니다.

5. 게시 슬롯은 DB가 나눠 줍니다. claim 함수가 행을 FOR UPDATE로 잠그고, 계정 advisory lock을 잡고, 하루 상한을 본 뒤 30~900초짜리 lease 토큰(잠시 빌려 쓰는 잠금 표식)을 줍니다. 앱 메모리 락 대신 이쪽을 택해 프로세스가 몇 개든 판정은 DB 한 곳에서 합니다.

6. 장부 하나로 정책 두 가지를 지킵니다. 직접 경로가 프록시 상한을 비켜 가던 구멍을 막으려고 LiteLLM과 Vertex 직접 경로가 같은 Redis 키에 비용을 더합니다. 장부를 못 읽으면 개별 호출은 통과, 반복 자동화 시작은 보류입니다. 읽기는 쓰기 권한 없는 Redis 사용자로 합니다.

7. 렌더 재시도는 마감 하나를 나눠 씁니다. LLM이 짠 모션 계획이 HyperFrames 검사에서 떨어지면 기본 템플릿으로 다시 렌더합니다. 두 시도가 마감 하나를 같이 써서, 재시도가 대기 시간을 두 배로 늘리지 않습니다. 렌더는 한 번에 하나씩 실행합니다.

8. 게시할 파일은 체크섬으로 고정합니다. 게시 직전에 Drive 파일의 mime, 크기, md5, 위치를 다시 봅니다. QA 관문은 ffprobe로 60초, 1080×1920, 100MB, h264를 확인하고 AI 영상은 프레임마다 AI 표시를 찾습니다. 업로드는 8MiB씩 읽고 308 응답(이어 올리기 신호)에서 이어 올립니다.

상태 · 판단 · 한계

- 이 신규 제작 경로는 기술은 완성됐지만 안에 쓰는 프롬프트 같은 세부는 아직 다듬는 중입니다. 이 경로로 만든 콘텐츠의 SNS 실게시 실적은 없고, 운영 canary는 MP4의 Drive 저장까지 확인했습니다(2026-10). 기존 영상은 캡션·해시태그만 계정별로 바꿔 여러 계정에 동시에 올리는 별도 경로로 이미 실제 게시되고 있습니다.

- 승인 만료는 매일 03:00 KST 크론이 반려, 이슈 취소, 행 닫기 순으로 처리합니다. 이슈 취소가 실패해도 행은 닫아 같은 실패가 반복되지 않습니다.

- 한계: 상품 사실 검증은 프롬프트 규칙뿐이고, 새 인스타 토큰 등록은 개발자 설정이 필요합니다. 이 경로는 아직 실게시가 없어 조회수 같은 성과는 측정하지 못했습니다.

- 다시 한다면: 가격과 구성 같은 상품 사실을 상품 DB와 대조하는 검사를 코드로 넣겠습니다. 지금은 모델이 규칙을 어기면 사람이 미리보기에서 잡아야 합니다.

내가 정한 것

- 단계 진행은 LLM 없는 상태기계가 맡고 LLM은 리서치, 대본, 프롬프트만 씁니다. 다음 단계를 모델에게 맡기면 재실행할 때마다 결과가 달라집니다.

- 형식 수리 1회와 기본 템플릿 재렌더 말고는 자동으로 다시 하지 않고 닫습니다. 비용이 들거나 밖에 공개되는 일은 사람이 다시 누릅니다.

- 게시 기본값은 수동이고, 하루 상한은 앱 설정이 아니라 DB 인덱스에 뒀습니다.

CS

벤더 CS 에이전트

StyleSeller · 사내 비공개 저장소

태그된 질문에만, 인계 없이 답하고, 숫자와 브랜드는 근거와 대조해 고쳐 쓰는 카톡 챗봇

기간	2026.06~현재
역할	설계: 수집 경로, 거래처 게이트, 근거 대조 규칙 / 구현: 서버 에이전트, kakaocli(맥 카톡 DB를 읽고 보내는 CLI) 포크의 발송·도착 확인, 윈도우 수집기 / 운영: 사고 대응, 09-02 응답 방식 전환. AI 코딩 도구를 썼고, 범위는 제가 정하고 결과는 시험과 실측으로 확인했습니다
저장소	서버(벤더 에이전트), kakaocli 포크, CS Copilot 확장 · 로컬 발송기

배경

직원 PC에서 대화를 모아 서버로 올리고, 답도 직원 맥에서 보냅니다. 서버 에이전트는 거래처별 기록과 LightRAG 지식 허브(개체·관계 그래프 RAG, 워크스페이스 13개)를 검색해 답을 씁니다. 2026-09-02부터 되묻기를 끄고 태그된 질문에만, 인계 없이 답합니다. 대신 금액·%·브랜드를 근거와 맞춰 보고, 어긋나면 다시 검색해 고쳐 씁니다.

구조와 규모

1. 기술

macOS kakaocli 포크(Swift), 윈도우 읽기 전용 수집기(Python), Python 3.12, Gemini Flash 계열(LiteLLM 경유), LightRAG 1.5.6, 구글 시트 · 드라이브

2. 규모 (항목별 기준일 표기)

- 벤더 에이전트 코드 · 테스트 · 설정 합 6.1만 줄, 커밋 600, 테스트 파일 107 (2026-09-23)
- CS Copilot 확장 · 로컬 발송기 커밋 631, 20,596줄, 테스트 56 (2026-09-23)
- 시험: 거래처 게이트 20, 근거 판정 16, 생존 표시 30, 맥 Swift 34, 윈도우 Python 72

3. 어떻게 동작하나

① 맥은 kakaocli가 카톡 DB에서 마지막 logId 뒤만 읽고, 윈도우는 읽기 전용 수집기가 올립니다. 서버가 2xx로 답해야 넘어갑니다.
② 서버는 15초 안에 다시 온 같은 메시지를 버립니다. 거래처별 원장 · 시트 · 드라이브에 쌓고, 하루 2회(09:30 · 18:10) 위키로 정리해 지식 허브에 넣습니다.
③ 방 이름의 거래처 ID나 이름을 업무툴 응대 대상 명단과 대조합니다. 명단에서 거래처를 못 찾으면 답하지 않습니다.
④ @회사_AI 태그가 붙은 질문만 받습니다. 정규식은 태그만 감지하고, 검색과 답 작성은 모델이 합니다. 다른 거래처 이름은 [다른 벤더]로 가립니다.
⑤ 초안의 금액 · % · 브랜드를 근거와 대조하고, 어긋나면 다시 검색해 고쳐 씁니다. 답은 직원 맥에서 보내고, 카톡 DB 개수가 늘었는지로 도착을 확인합니다.

사고와 수정

1. 규칙이 판정을 가로챔(~2026-08-30). 트리거용 정규식이 차단까지 맡고 있어서 일부 메시지가 LLM 판정 없이 조용히 막혔습니다. 정규식은 태그 감지만 하게 줄이고, 답할지 판단은 모델로 옮겼습니다.

2. 발송이 끝나지 않음(2026-08-17~18). macOS 15에서 카톡 창이 백그라운드에 있으면 접근성(AX) 호출이 시간 제한을 무시하고 멈춰서 발송 147건 중 31건(21%)이 영구 실패로 남았습니다. 호출을 별도 스레드로 빼고 세마포어로 3초가 넘으면 기다림을 끊게 했습니다. 09-08부터는 보낸 뒤 도착을 다시 확인합니다.

3. 과거 대화 홍수(2026-09-16). 옛 판 수집기가 과거 대화를 통째로 올리는 바람에 윈도우 PC 3대가 50분에 2,500건을 보냈고, 시트 API가 429로 막혀 뒤에 설치한 2명은 행이 한 시간 넘게 비었습니다. 서버가 기준일 이전 기록은 버리고 200으로 답하게 했습니다. 5xx로 답하면 수집기가 같은 기록을 또 올립니다.

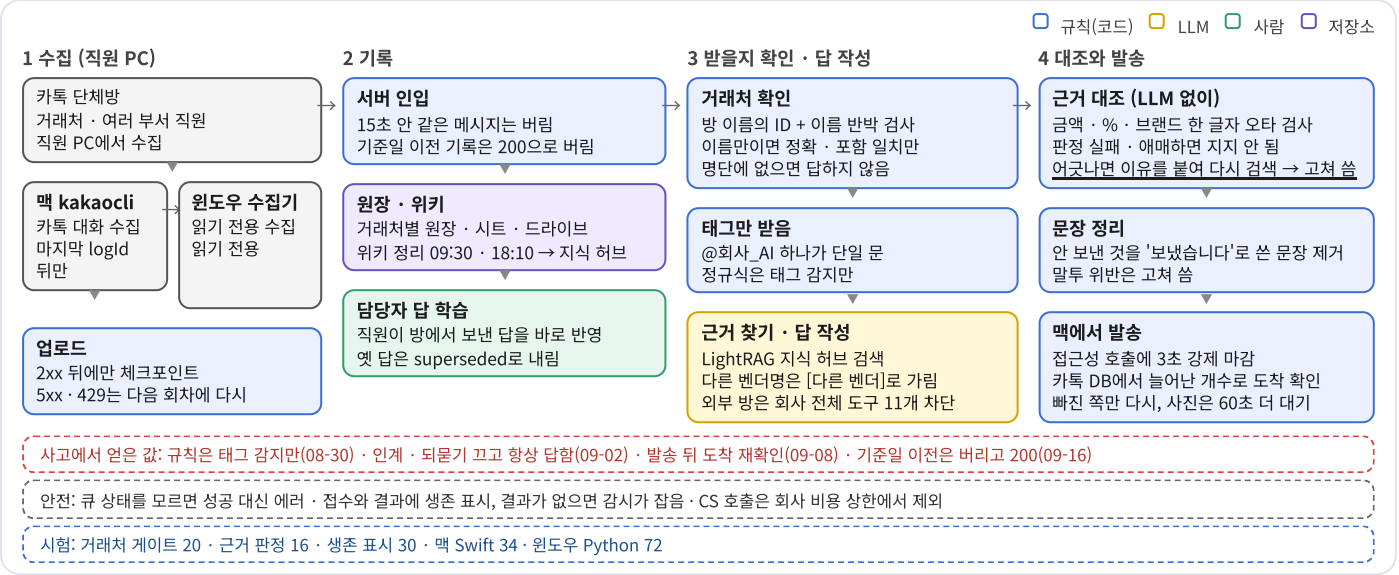
상태 · 판단 · 한계

- 2026.06부터 수집, 원장 · 위키 정리, 태그 질문 답변이 돌고 있습니다. 담당자가 방에서 직접 보낸 답은 바로 학습하고, 같은 질문의 옛 답은 superseded(새 답으로 대체됨)로 내립니다.
- 벤더 · 인플루언서 CS 호출은 회사 하루 비용 상한에서 빠져 있습니다. 윈도우 수집기는 자동 갱신기가 없어 새 판을 재설치로 배포합니다.
- 다시 한다면: 윈도우 수집기 첫 판부터 자동 갱신과 기준일을 넣겠습니다. 09-16 과거 대화 홍수는 옛 판 수집기 탓이었고, 재설치 말고는 바로잡을 길이 없었습니다.

내가 정한 것

- 2026-09-02에 되묻기를 끄고, 태그된 질문에만 인계 없이 답하게 바꿨습니다. 그 대신 숫자 · 브랜드 대조는 LLM에 맡기지 않고 코드로 합니다. 옛 인계 분기는 테스트가 지키도록 코드에 남겼습니다.
- 정규식은 태그 감지만 맡기고 판정은 모델에게 넘겼습니다. 정규식이 차단까지 하던 방식은 2026-08-30에 걷어냈습니다.
- 거래처 확인에서 퍼지 매칭을 뺐습니다. 이름이 비슷하다는 이유로 남의 방에 답하는 것보다 답하지 않는 쪽이 낫다고 판단했습니다.

Architecture



직원 PC 수집 → 서버 원장 · 위키 → 지식 허브 → 태그 질문 답 작성 → 근거 대조 · 재작성 → 맥 발송 · 도착 확인

기술 하이라이트

1. 비슷한 이름으로는 거래처를 정하지 않습니다. 방 이름 앞 '(1234)'를 ID로 읽습니다. 이름 부분이 다른 벤더를 자모 유사도 0.80 이상으로 더 잘 가리키면 ID 오타로 보고 막습니다. ID가 없으면 정확히 같거나 포함 관계인 이름 하나만 인정합니다. 흔한 접미어만 겹쳐도 0.81~0.88이 나와서 퍼지 매칭은 버렸습니다.

2. 메시지 1건이 51번 들어왔습니다. kakaocli는 안 읽은 방의 최신 메시지를 2초마다 다시 보냅니다. 세어 보니 1건에 51회, 분당 170~200건이었습니다. (방, 메시지, 안읽음 수, 내가 보냄) 키로 15초 안의 재전송을 버립니다. 안읽음 수가 키에 있어서 방을 닫고 다시 연 신호는 통과합니다.

3. 숫자와 브랜드는 코드로 대조합니다. 초안의 금액과 %를 정규화합니다('3만원'은 30000원). 검색 근거에 없는 값, 근거 브랜드와 한 글자만 다른 단어를 위반으로 잡습니다. 위반이 나오면 이유를 붙여 다시 검색하고 고쳐 씁니다.

4. '보냈다'는 사실일 때만 씁니다. 그 턴에 발송 API가 안 불렸으면 '제안서 보내드렸습니다' 같은 완료형 문장은 지웁니다. 같은 idempotency key로 재요청이 오면 실제 큐 상태를 봅니다. 처리 중이거나 보낸 건은 다시 넣지 않고 실패한 건만 재시도합니다. 상태를 모르면 성공 대신 에러를 냅니다.

5. 다른 거래처 사례는 가립니다. 검색 결과에 지금 거래처가 아닌 벤더명이 나오면 [다른 벤더]로 바꾸고, 바꿨다는 사실을 모델에 알립니다. 회사 전체를 조회하는 Spark 도구 11개는 외부 방에서 모두 막습니다. 상품 추천 조회에 넣는 벤더명은 모델이 아니라 서버가 정합니다.

6. 도착은 카톡 DB 개수로 셉니다. 보내기 전에 기준 개수를 잡아 둡니다. 보낸 뒤 로컬 DB에서 글과 사진이 몇 개 늘었는지 세고, 빠진 쪽만 다시 보냅니다. 사진은 60초를 더 기다립니다. 70명 방에서는 같은 사진이 두 번 가는 쪽이 한 장 빠지는 쪽보다 나쁘다고 봐서입니다. 서버에 완료를 알리는 것도 이 확인 뒤입니다.

7. 체크포인트는 2xx 뒤에만 옮깁니다. 업로드기는 2xx를 받은 뒤에만 오프셋을 옮깁니다. 422 · 413은 건너뛰고, 5xx · 429면 커밋 없이 멈췄다가 다음 회차에 다시 올립니다.

8. 처리 경로가 스스로 생존 표시를 남깁니다. 태그 메시지를 접수할 때 한 번, 결과에 닿았을 때 한 번 표시를 남깁니다. 업무시간 기준 임계값을 넘도록 결과 표시가 없으면 감시가 잡아냅니다. 감시가 밖에서 '초안이 없으면 멈춘 것'으로 추정하던 방식은 리뷰에서 뺐습니다. 이 경로의 시험은 30개입니다.

플랫폼 에이전트 플랫폼 · Spark

누가 말하든 일은 CEO 에이전트의 작업지시서로 시작하고, 위임과 도구 실행은 서버가 통제합니다

기간	2026.05~현재(착수 05-17, Spark·Work OS 설계 07-18~19)
역할	설계: 작업지시서, 위임 가드, 도구 게이트 구조. 구현: 공용 도구·스킬 상자 연결, 승인 1회, LiteLLM 상한, Jev 라우터 연결. 검증·운영: 평가 세트 작성과 VM 배포. AI 코딩 도구를 썼고, 범위는 제가 정하고 결과는 직접 확인했습니다
저장소	사내 저장소(Paperclip 포크), 스킬 폴더, 서비스 코드

배경

에이전트가 서로 이슈를 넘기며 일하는 사내 플랫폼입니다. Paperclip(에이전트 이슈 관리 오픈소스)을 포크했고, 회사 기능은 코어 밖 서비스 폴더 아래 HTTP 서비스로 붙였습니다. 예외는 Work OS(업무 상태·승인·실행 기록 앱)와 Spark뿐입니다. 직원은 Spark로 업무 DB 4개를 읽기 전용으로 묻고, 일은 CEO 에이전트에게 맡깁니다. 위임이 길어지면 요청이 바뀌고 도구 하나가 돈이나 외부 노출로 이어지니, 통제는 프롬프트가 아니라 서버 코드에 뒀습니다.

구조와 규모

1. 기술

Paperclip 포크(Express 5, Drizzle, PostgreSQL), MCP SDK 1.29, Hermes(에이전트 실행 런타임), Python 3.12와 FastAPI, LiteLLM, Redis, Langfuse 4.50, Gemini Flash 계열(LiteLLM 경유)

2. 규모 (2026-10-24 기준)

- 본인 커밋 약 5,000(merge·업스트림 제외), 테스트 파일 약 2,200 (2026-10-24)
- 공용 스킬 109개, 트리거 평가 파일 66개(문장 1,282개), 도구 매니페스트 67개

3. 어떻게 동작하나

① Spark, Slack, 브랜드툴 요청은 모두 CEO 에이전트로 갑니다. 구조화된 브랜드툴 요청만 콘텐츠 오케스트레이터로 직행하고, 지시서 기록은 남깁니다.

② CEO가 작업지시서 v1을 씁니다. 원문 첨부와 sha256, 목표, 완료 기준, 제약, 승인 여부가 들어갑니다.

③ CEO 직속 Hermes 디스패처가 부서장 10곳 이하 중 한 곳에 맡기고, 끄면 CEO 직접 위임으로 돌아갑니다. 서버는 위임 깊이를 2, 지시서당 위임을 5회로 제한하고 A→B→A 되돌림을 막습니다.

④ BM25가 공용 스킬(운영 적용 시 115개) 중 후보를 추리면 Jev가 게이트·적합도로 다시 거르고, Gemini가 그 중에서 스킬 1개를 고릅니다. Jev가 실패하거나 시간을 넘기면 Jev 단계를 건너뛰고 Gemini가 전체 목록에서 바로 고릅니다. 고도화 과정은 13쪽에 자세히 적었습니다.

⑤ 도구는 6단계 게이트와 사람 승인을 거쳐 실행됩니다. 게시는 Publisher 관문(게시 전용 단일 서비스)이 맡고, 기본은 수동이며 계정당 하루 1건을 DB가 막습니다.

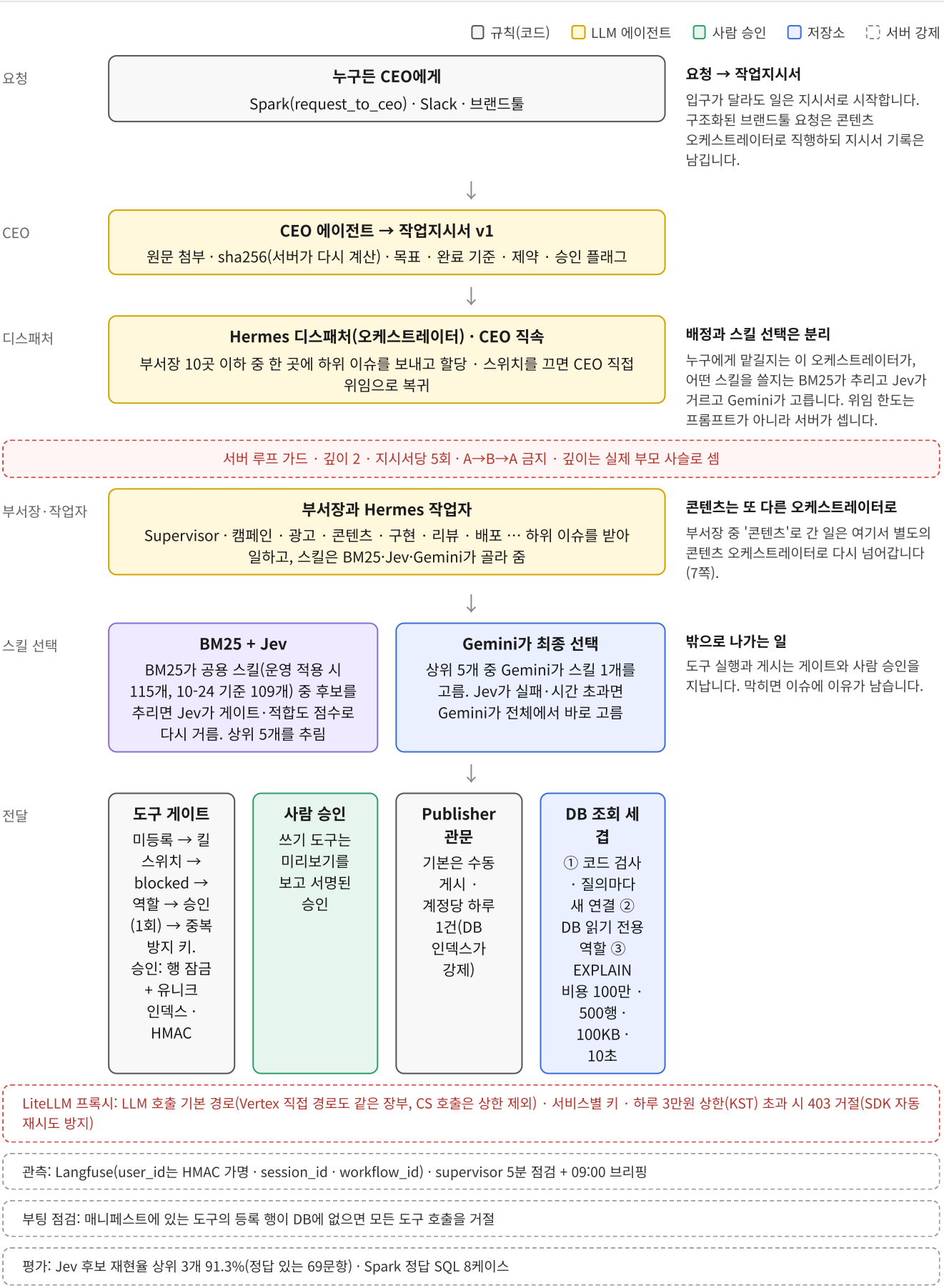
사고와 수정

1. Spark 전체 닫힘(2026-10-24). 새 Spark 도구의 DB 등록 행이 빠져 부팅 점검이 실패했고, Spark 호출이 전부 거절됐습니다. 점검이 실패하면 막도록 만든 그대로 동작했습니다. 재시작으로 복구한 뒤 도구마다 등록 행을 마이그레이션으로 넣는 규칙과 회귀 시험을 붙였습니다.

2. 렌더 중인 작업 회수(2026-10-24). 서버 회수기가 5분 넘은 영상 작업을 멈춘 것으로 보고 회수했고, 7분 40초 뒤 나온 결과는 409로 거절됐습니다. 120초마다 생존 신호를 보내되 이슈 마감이 지나면 신호를 끊게 해, 정말 멈춘 작업은 여전히 회수됩니다. 운영에서 신호 수신(200)과 회수 0건을 확인했습니다.

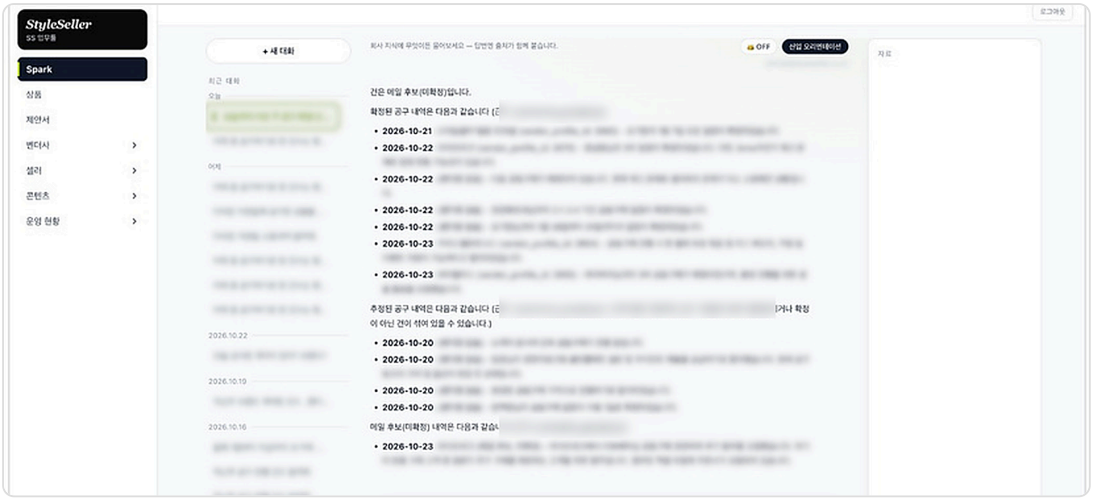
3. Jev 라우터 경량화(2026-10-23). 속도와 가벼움을 위해 Jev 라우터에도 경량 모델을 썼습니다. 응답 형식만 맞추는 어댑터를 두고 기존 라우터 코드는 그대로 뒀습니다.

Architecture



요청 → CEO 작업지시서 → Hermes 디스패처 → 부서장·작업자(BM25+Jev→Gemini 선택) → 도구 게이트·사람 승인 → Publisher, LLM 호출은 기본적으로 LiteLLM 경유

화면



Spark 답변 화면. 질문 제목과 상대 이름은 가렸고, 답마다 근거 도구 이름이 붙습니다

기술 하이라이트

1. 작업지시서 원문은 해시로 잠급니다. 위임이 단계를 거칠수록 요청 문장이 바뀌니 원문을 NFC 정규화하고 줄바꿈을 맞춘 뒤 sha256과 함께 잠급니다. 클라이언트가 보낸 해시는 서버가 덮어씁니다. 중간 요약 대신 원문을 넘길 때 supervisor 오류가 줄었다는 외부 벤치마크를 따랐습니다.

2. 위임 루프는 서버가 끊습니다. 플러그인은 이슈가 생긴 뒤에 지켜볼 수만 있어 서버 코어를 고쳤습니다. 깊이 2, 지시서당 5회, A→B→A 금지입니다. 깊이는 값을 바꿀 수 있는 열 대신 실제 부모 이슈를 거슬러 올라가며 셉니다. 6번째 위임과 깊이 위조는 테스트에서 거절됩니다.

3. BM25가 추리고 Jev가 거르고 Gemini가 고릅니다. BM25가 115개 중 후보를 추리면 Jev가 게이트·적합도로 다시 거르고, Gemini가 그 중 1개를 고릅니다. 실패, 시간 초과, 차단기(2회 실패 후 300초) 중 하나면 Jev 단계를 건너뛰고 Gemini가 전체 목록에서 바로 고릅니다. 롤아웃 대상은 sha256 앞 8자리로 나눕니다.

4. 승인은 정확히 한 번만 씁니다. 승인 행을 FOR UPDATE로 잠그고, 부분 유니크 인덱스가 한 번 더 막습니다. 실행이 실패해도 승인은 소진됩니다. 중복 방지 키는 ON CONFLICT DO NOTHING 뒤 재조회라 경쟁이 없습니다. 동시 요청 10개에 실행 1개를 테스트로 확인했습니다.

5. 쓰기 승인은 미리보기에 서명합니다. 입력 해시, 미리보기 해시, 요청자, 실행 ID, 만료 시각을 정규 JSON으로 묶어 HMAC-SHA256으로 서명하니 DB 쓰기 권한이 있어도 키 없이는 통과하지 못합니다. 중복 방지 키에는 시각을 넣지 않아 재시도해도 승인이 늘지 않습니다.

6. Spark DB 조회는 질의마다 새 연결을 엽니다. 읽기 전용 역할만으론 부족했습니다. set_config 한 번이면 타임아웃과 읽기 전용이 풀리고, 재사용 연결이 그 상태를 다음 사람에게 넘깁니다. 위험 함수 23개와 세미콜론, 쓰기 CTE를 거른 뒤 연결 1개를 열고 끝나면 닫습니다.

상태 · 판단 · 한계

- 운영 중입니다. 도구 실행 전사 스위치는 기본 꺼짐이고 도구마다 따로 켭니다. Langfuse에는 user_id를 HMAC 가명으로 바꿔 보냅니다.
- 평가(측정 당시 모델 기준): 정답 있는 69문항에서 Jev 후보 재현율(정답이 후보 안에 든 비율)은 1위 85.5%, 상위 3개 91.3%입니다(2026-10-24).
- 다시 한다면: 매니페스트와 도구 등록 행을 CI에서 대조하겠습니다. 10-24 장애는 부팅에서야 드러났는데, 병합 전에 잡을 수 있던 문제였습니다.

내가 정한 것

- 역할별 스킬 묶음 대신 전체 색인을 택했습니다. 57건 선택 세트에서 두 방식의 정확도는 0.947로 같았습니다(p=1.0). 다만 묶음 방식은 이름을 불러야만 쓰는 스킬을 놓쳤습니다. 66건 비용 차이는 27원이었습니다.
- 배정은 디스패처가, 스킬 선택은 BM25가 추리고 Jev가 거르고 Gemini가 고릅니다. Jev가 실패해도 Gemini가 전체 목록에서 바로 골라 일이 멈추지 않습니다. 선택은 작업자가 아니라 항상 Gemini가 합니다.
- 루프 가드, 승인 1회, 유료 상한 같은 통제는 모델이 어겨도 막히도록 모두 서버와 DB에 둡니다.

후속 · Jev 스킬 라우터 고도화 (다음 쪽)

- Spark의 BM25 힌트와는 별개로, 같은 Jev 규칙을 쓰는 라우터를 손봐 새 질문 132건에서 최종 정답률 90.15%를 쟀습니다. 위 재현율과는 다른 지표입니다.
- 이 방식은 Spark의 실행 런타임인 Hermes 에이전트에 반영했습니다.
- 정확도보다는 정답표 오류, 가짜 스킬, 부풀려진 점수를 걸러낸 측정이 핵심이었습니다.

Jev

Jev 스킬 라우터 고도화

Spark 사례의 후속 · 2026.10.25 ~ 10.30
StyleSeller · 사내 비공개 저장소

정확도를 올리려다, 먼저 '재는 방법'이 틀렸다는 걸 찾았습니다

Spark 사례의 후속입니다. Jev는 텍스트 대신 확률을 돌려주는 판정 모델(TypeSafe)이고 Jev 라우터는 후보 추림부터 심사까지 전체입니다. Spark의 실행 런타임인 Hermes가 작업을 배정한 하위 에이전트들이 Jev 라우터로 쓸 스킬을 고릅니다. 이번에 손본 LLM 제안자 방식을 여기에 반영했습니다(앞 사례의 BM25 추림은 10-24 기준). 스킬 목록은 10-24의 109개에서 시험 때 115개로 늘었습니다. 시험 설계, 실험, 결과 확인, 반영 결정은 제가 했고 구현은 AI 코딩 도구를 쓰되 범위와 결과는 직접 확인했습니다.

90% 부근
정답률 90.15% (132건 중 119건, 1건=0.76%p)

숫자가 네 번 바뀜
정답표 오류, 부풀려진 점수, 가짜 스킬, 재측정

운영 반영
전면 적용, 실제 질문 3/3 정답

요청 → BM25+Jev: 후보 추림·검증(게이트 → fits) → Gemini: 최종 선택 → 스킬 1개 또는 '없음'

처음엔 BM25 없이 판정 모델만 쓰면 45~53%가 한계였고 10-26에 LLM 제안자를 넣어 80%대가 됐습니다. 이번엔 그 위에서 10-27~30에 90%를 겨눴습니다. 앞 사례의 91.3%는 BM25 후보 재현율이고 시험지도 달라 비교하지 않습니다.

정정과 재측정

정답표가 틀려 있었습니다 10-27
'필요 없는 질문에 불렀다'는 오탐 6건 중 5건은 라우터가 맞았습니다. 직접 열어 보지 않았다면 오판했을 겁니다. 고치기 전후 숫자는 모두 남겼습니다.

본 시험지로 고른 설정은 부풀려 보였습니다 10-27 · 10-29
봤던 시험지 +4.4%p, 처음 본 132건 +1.2%p였고 정정 후 재측정에서는 0과 +1.5%p였습니다. 이득이 작고 흔들려 같은 시험지 점수는 근거로 쓰지 않았습니다.

시험 폴더에 가짜 스킬이 섞여 있었습니다 10-25부터 섞임 · 10-27 저녁 발견
같은 오답이 두 시험지에서 나와 저는 "진짜 결함"이라고 두 번 말했습니다. 운영 저장소 115개와 하나씩 맞춰 보니 116번째 스킬은 시험 폴더에만 있었습니다. 운영에서는 생길 수 없는 오류라 결론을 철회했습니다.

정정한 뒤 다시 돌렸더니 또 달라졌습니다 10-29
교차 검토에서 "비교 대상도 오염된 데이터로 쟀다"는 지적이 나와 가짜 스킬을 3곳에서 모두 치우고 222건(90 + 132)을 다시 돌렸습니다. 켜지 않은 비교 설정은 확신만 높으면 적합도가 0.22밖에 안 돼도 그 스킬을 골라 틀렸습니다. 운영에 켠 방식은 한 번도 그러지 않았고 그래서 비교 설정은 켜지 않았습니다.

무엇을 쟀나	운영 방식	켜지 않은 비교 설정
① 첫 측정 · 정답 있는 60건 (정답표 오류 섞임, 10-27)	83.3%	91.7%
② 정답표 정정 후 · 90문항 (10-27)	88.9%	93.3%
③ 가짜 스킬 제거 재측정 · 봤던 시험지 90문항 (10-29)	94.4%	94.4%
④ 같은 재측정 · 처음 본 132건, 이후 재사용 (10-29)	**90.15%**	91.7%

운영에 들어갔습니다. BM25가 후보를 추리면 Jev가 게이트와 fits(후보별 적합도)로 다시 걸러 상위 5개만 남기고, Gemini가 그중에서 최종 1개를 고릅니다. 비교 설정은 스위치 2개를 더 켠 것인데, 이득이 +1.5%p로 작고 적합도가 낮은 스킬로 우회하는 오답 위험이 있어 켜지 않았습니다.

운영 적용과 실험

- **BM25+Jev로 먼저 거르면 Gemini 비용이 약 1/10입니다.** BM25가 115개 중 후보를 추리고 Jev가 게이트·적합도로 다시 걸러 상위 5개만 Gemini에 넘기는 방식을 작은 시험으로 확인한 뒤 LLM 제안자 측정에 쓴 같은 새 질문 132건으로 다시 돌렸습니다. Gemini가 읽는 스킬이 115개에서 5개로 줄어 비용이 약 1/10이고, 정확도는 132건 중 119건, 90.15%로 Gemini가 115개를 모두 읽는 방식과 비슷했습니다. Jev는 글을 생성하지 않고 확률만 돌려줘서 Gemini보다 빠르고 저렴합니다(TypeSafe 공개 기준 응답 70~500ms, 제 실측 중앙값 360ms). 앞에서 싼 모델로 추리고 뒤에서 고르는 이 구조는 공개 연구 SkillRouter(arXiv 2603.22455)와 같은 방향입니다. 이 방식을 전면 적용해 지금 Hermes가 작업을 배정한 하위 에이전트가 스킬을 고를 때 씁니다. Jev가 실패하거나 시간을 넘기면 Gemini가 전체 목록에서 바로 고릅니다. 근거와 범위: 스킬 115개, 처음 본 질문 132건 기준이고, 비용 1/10은 Gemini가 읽는 스킬 수로 어림한 공개 단가 기준 추정입니다.
- **쿼터 안에서 실험했습니다.** 저장한 답으로 호출 없이 설정을 비교하고 20문항 → 90문항 → 132건 순으로 늘렸습니다. 모든 시험 기록에서 한도 초과(429)는 0건이었습니다.

켠 것 · 제가 정한 것

코드를 바꾼 곳 3군데를 서로 비교해 차이가 없는지 확인하고, 기존 설정 파일을 백업한 뒤 설정을 한 줄 추가했습니다. 서버를 재시작해 정상 작동을 확인하고, 실제 질문 3개를 넣어 3개 모두 정답인 것까지 본 뒤에야 BM25가 후보를 추리고 Jev가 다시 거른 뒤 Gemini가 최종으로 고르는 방식을 운영 전체에 적용했습니다. 이 방식을 켤지는 결과를 보기 전에 기준부터 정해뒀습니다 — 처음 보는 질문에서 115개를 모두 읽는 방식보다 정확도가 떨어지지 않고, 엉뚱한 스킬을 고르는 일이 늘지 않으며, 비용이 줄 때만 켠다는 기준이었습니다.

켜지 않은 것 · 한계

스위치 2개를 더 켜는 비교 설정도 함께 시험했지만 운영에는 넣지 않았습니다. 정답률이 1.5%포인트 정도밖에 오르지 않아 효과가 작았고, 모델이 확신만 높으면 적합도 점수가 0.22처럼 낮아도 그 스킬을 그대로 통과시켜 버리는 위험이 확인됐기 때문입니다.

이 실험 결과에도 한계가 있습니다. 여기 적힌 숫자는 시험 환경에서 나온 값이라 실제 운영 전체 트래픽의 정답률과 똑같다고 보장할 수 없습니다. 시험 문제 중에는 정답이 하나가 아니라 둘 다 맞는 애매한 문제도 섞여 있습니다. 그리고 스킬 목록에 새 스킬이 추가될 때마다 시험 문제의 정답지도 같이 고쳐야 하는데 그 절차가 없어서, 정답지 자체에 오류가 10건(기존 시험지 5건 + 새 질문 5건) 생겼습니다. 이 오류를 자동으로 잡아내는 점검 장치는 아이디어만 냈고 아직 만들지는 않았습니다.

지식 허브

LightRAG 지식 허브 · 메일 에이전트

StyleSeller · 사내 비공개 저장소

에이전트 4곳이 같은 지식 허브에서 근거를 찾고, 메일은 근거 있는 초안만 직원 초안함에 갑니다

기간 2026.03.31~현재
역할 설계: 워크스페이스 등록부, 검색 계획과 근거 판정 단계, 초안 검증 순서 / 구현: 메일 에이전트 서버, 검사기, 보낸 메일 학습 커서 / 운영: 초안함 배포, 사고 분석과 복구
저장소 메일 에이전트

배경

벤더 CS, 인플루언서 CS, Spark, 판매 파트너사 매칭이 LightRAG 지식 허브(워크스페이스 13개) 하나를 같이 씁니다. 질문마다 자료를 먼저 찾아 붙여 답하는 RAG 저장소입니다. 워크스페이스마다 설명·주인·채널·민감도가 붙어 있고, 모르는 워크스페이스에 쓰려 하면 바로 오류가 납니다. 공용 메일함 문의에는 근거를 붙인 답장 초안을 담당자 Gmail 초안함에 넣습니다. 보내는 일은 직원이 합니다.

구조와 규모

1. 기술

Python 3.12·FastAPI·Pydantic v2, LightRAG 1.5.6(Postgres/pgvector + Neo4j 5), RAG-Anything/MinerU, FlashRank(가벼운 리랭크 모델), Gemini Flash 계열(LiteLLM 경유), Gmail, pytest

2. 규모 (2026-10-24 기준)

- 커밋 약 1,470 · 제품 코드 약 47,000줄 · 테스트 약 33,600줄 (10-24)
- 라우터 27개 · 테스트 함수 약 1,750 · 워크스페이스 13개 (10-24)
- 검색 평가 세트 18건(질의·기대 워크스페이스·필수 근거·금지 주장)

3. 어떻게 동작하나

① LLM 계획기(temperature 0)가 워크스페이스 설명과 데이터 목록을 보고 질의를 최대 12개 세웁니다. 검증에 실패한 계획은 버리고 규칙 라우터가 대신합니다.

② 검색 모드(벡터와 그래프를 함께 쓰는 mix)는 호출 지점마다 코드가 고정합니다. 에이전트 검색은 top_k(가져올 결과 수) 8이고 같은 질의는 60초 캐시합니다.

③ 근거는 규칙으로 판정합니다. 가격·재고·정산 같은 실시간 값은 금지 주장이고, 환불 확정·계약·대량 발송에는 사람 승인 표시가 붙습니다. 모자란 근거 항목만 한 번 더 찾습니다.

④ 리랭크(결과 순서를 다시 매기는 단계) 점수로 주제별로 묶습니다. 묶음 안에서는 공식 정정과 최신 문서를 앞에 두고, 결과는 묶음마다 하나씩 번갈아 내보냅니다.

⑤ 메일 초안의 주장 최대 6개를 검사기와 LLM 검증기가 근거와 대조합니다. 근거가 없으면 다시 쓰고, 그래도 남으면 초안을 버립니다.

사고와 수정

1. 리랭크 누락(2026-06-19~07-23). 06-18 에이전틱 RAG(LLM이 검색 계획을 세우는 방식)로 바꾼 다음 날부터 CEO 메일 초안의 품질이 떨어졌습니다. 커밋 기록을 거슬러 올라가 보니 전환 때 리랭크 단계가 빠져 있어서 07-23에 되살렸습니다.

2. 검색 과부하(07-10, 08-12). 인플루언서 CS 봇이 공유 연결 풀을 다 잡아 매칭 검색까지 500이 났습니다(07-10). top_k 40 검색은 Neo4j 과부하 경고를 냈습니다(08-12 분석). 연결 풀은 워크스페이스별로 나누고 에이전트 검색은 top_k 8로 묶었습니다.

3. 조용한 그래프 장애(2026-10-04). Neo4j가 멈춰도 컨테이너는 정상으로 보였습니다. 이제 /readyz가 Neo4j에 RETURN 1을 2.5초 제한으로 보내 ok·fail을 표시하되, 이 실패만으로 503을 주지는 않아 재시작이 반복되지 않습니다. 검색은 2회 시도 뒤 503과 Retry-After 30으로 답합니다.

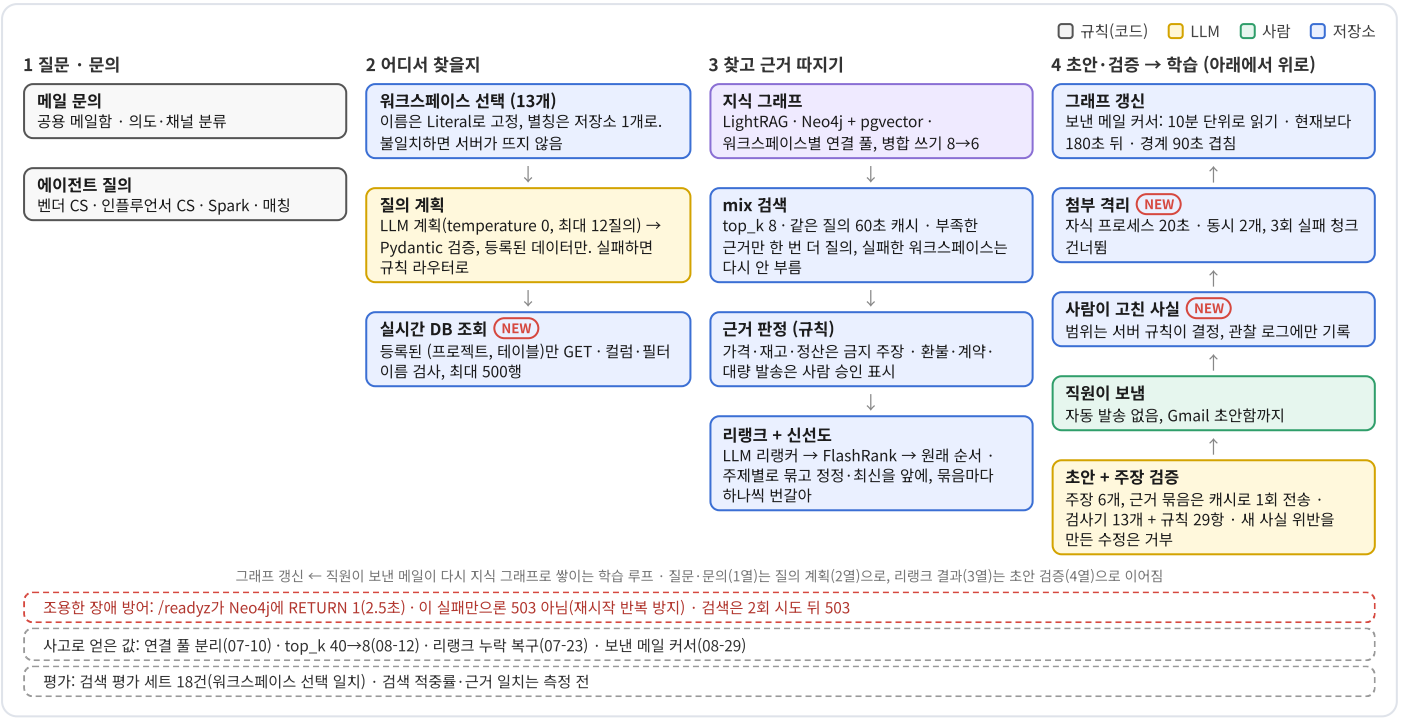
상태 · 판단 · 한계

- 답장 초안은 담당자 Gmail 초안함까지만 갑니다. 발송은 직원 몫이고, 2026-08-18부터 pytest를 통과한 이미지로만 배포합니다.
- 한계: 검색 평가 18건은 워크스페이스 선택만 봅니다. 검색 적중률, 근거 일치, 리랭크 복원 뒤 효과는 측정하지 않았습니다.
- 다시 한다면: 평가 세트에 근거 일치 채점을 먼저 넣겠습니다. 그랬다면 리랭크 누락을 초안 품질이 떨어지기 전에 잡았을 겁니다.

내가 정한 것

- 검색할 곳은 LLM 계획기가, 검색 모드는 코드가 정합니다. 에이전트가 모드까지 고르게 하지 않았습니다.
- 사실의 원천은 직원이 실제로 보낸 메일뿐입니다. AI 초안은 지식 허브에 다시 넣지 않습니다.
- 사람이 고친 사실은 바로 쓰지 않고, 관찰 로그에 먼저 쌓아 범위 판단을 확인합니다.

Architecture



보낸 메일 사실 · 카톡 위키 · PDF → 지식 허브(개체 · 관계) → LLM 계획기 → 검색 · 근거 판정 → 초안 · 답변

기술 하이라이트

1. 워크스페이스 이름이 어긋나면 서버가 뜨지 않습니다. 에이전트마다 이름을 달리 부르면 쓰기가 엉뚱한 저장소로 갈 수 있으니 정식 이름 13개는 Literal 타입으로 고정하고 별칭 약 50개는 저장소 하나로 모읍니다. 모듈을 불러올 때 누락이나 저장소 불일치가 보이면 RuntimeError로 기동을 멈춥니다.

2. LLM 검색 계획은 검증을 통과해야 씁니다. 계획은 Pydantic으로 검사합니다. 실시간 DB 조회는 등록된 (프로젝트, 테이블) 조합만 허용하고, GET만 쓰며 최대 500행입니다. 호출·파싱·허용 목록 중 하나만 실패해도 규칙 라우터로 돌아가며 이유는 fallback:<예외명>으로 남깁니다.

3. 리랭크가 실패하면 두 단계 더 대신합니다. 문서를 20개로 자른 뒤 LLM 리랭커가 먼저 실행합니다. 실패하면 FlashRank, 그것도 실패하면 원래 순서에 점수를 매겨 결과는 늘 나옵니다. 검색 단계 동시 실행은 3으로 낮췄습니다. 그 덕에 병합 단계의 Neo4j 동시 쓰기가 문서당 8개에서 6개로 줄었습니다.

4. 초안 검사기 13개가 오류 패턴을 하나씩 맡습니다. 요청 방향 반전, 없던 이전 메일 언급, 확정 조건 되묻기, 제안가를 확정가처럼 쓰기 같은 패턴이며 검사기마다 08-21~08-23 실제 메일 재현 사례를 넣었습니다. 처음엔 정규식이었는데 오탐이 이어져, 판정은 LLM 검사기로 옮겼습니다.

5. 새 거짓말을 만든 수정은 받지 않습니다. 위반 점수 합만 비교하면 2점짜리 사실 위반이 10점짜리 금지어를 고친 대가로 통과합니다. 고쳐 쓴 초안에 없던 사실 위반이 새로 생기면 다른 위반을 고쳤어도 그 수정을 버립니다. 고쳐도 남는 위반은 종류별로 "Draft rejected"를 남기고 폐기합니다.

6. 근거 묶음은 한 번만 보냅니다. 주장 6개를 검증할 때마다 최대 15만 자짜리 근거 묶음을 다시 보냈습니다. 지금은 묶음을 sha256 키로 5분 캐시해 다시 씁니다. 캐시가 실패하면 원래 방식으로 돌아가므로 판정 결과는 달라지지 않습니다.

7. 보낸 메일 커서는 Gmail 지연까지 셈합니다. 배포 때 보낸 메일 1,943건 중 48건이 학습에서 빠졌습니다. 지금은 10분 단위로 읽고 단위마다 커서를 저장합니다. 검색 인덱스가 늦게 잡히니 커서는 현재보다 180초 뒤에 두고, 경계는 90초 겹쳐 읽습니다. 500건이 넘는 단위에서는 커서를 밀지 않습니다.

8. 직원이 고친 사실은 서버가 범위를 정합니다. 회사 전체의 사실인지 거래 하나의 사실인지를 서버 규칙으로 판단해, 거래 하나의 사실을 파트너 전체로 넓히지 않습니다. 결과는 추가 전용 관찰 로그에만 쌓고, 초안과 검색에는 쓰지 않는 섀도 모드로 돌립니다.

업무툴

사내 업무툴 · 가격·매칭·맞춤카톡

StyleSeller · 사내 비공개 저장소

금액은 코드가 계산하고, LLM은 후보와 고칠 칸만 돌려줍니다

기간	2026.03 말~현재
역할	설계: 가격 계산 구조, 매칭 후보 합치는 규칙, 발송 승인 기준 / 구현: 가격 코드, 카탈로그 추출, 매칭, 발송 자동화, PC 워커 / 검증·운영: 시트 대조 시험, 발송·워커 운영 (셀러 추천 로직은 동료 CRM)
저장소	업무툴, 서버(직원 PC 워커)

배경

가격 계산은 구글 시트 수식에 있었고, 제안서마다 손으로 맞췄습니다. 이 수식을 코드로 옮기면서 실제 상품의 시트 값과 한 칸씩 대조하는 시험을 붙였습니다. 카탈로그 상품 추출과 판매 파트너사 매칭에서 LLM은 후보만 내고 등록과 검토는 직원 몫입니다. 맞춤카톡의 셀러별 추천은 동료가 만든 CRM이 정하고, 저는 그 대상을 받아 발송하는 자동화를 만들었습니다.

구조와 규모

1. 기술

React 19.2·Tailwind 4·Vitest 4, Postgres+PostgREST(테이블을 바로 여는 REST 서버), Gemini Flash 계열(LiteLLM 경유), 지식 허브 RAG 검색(자료를 찾아 답에 붙임), kakaocli, 직원 PC 워커

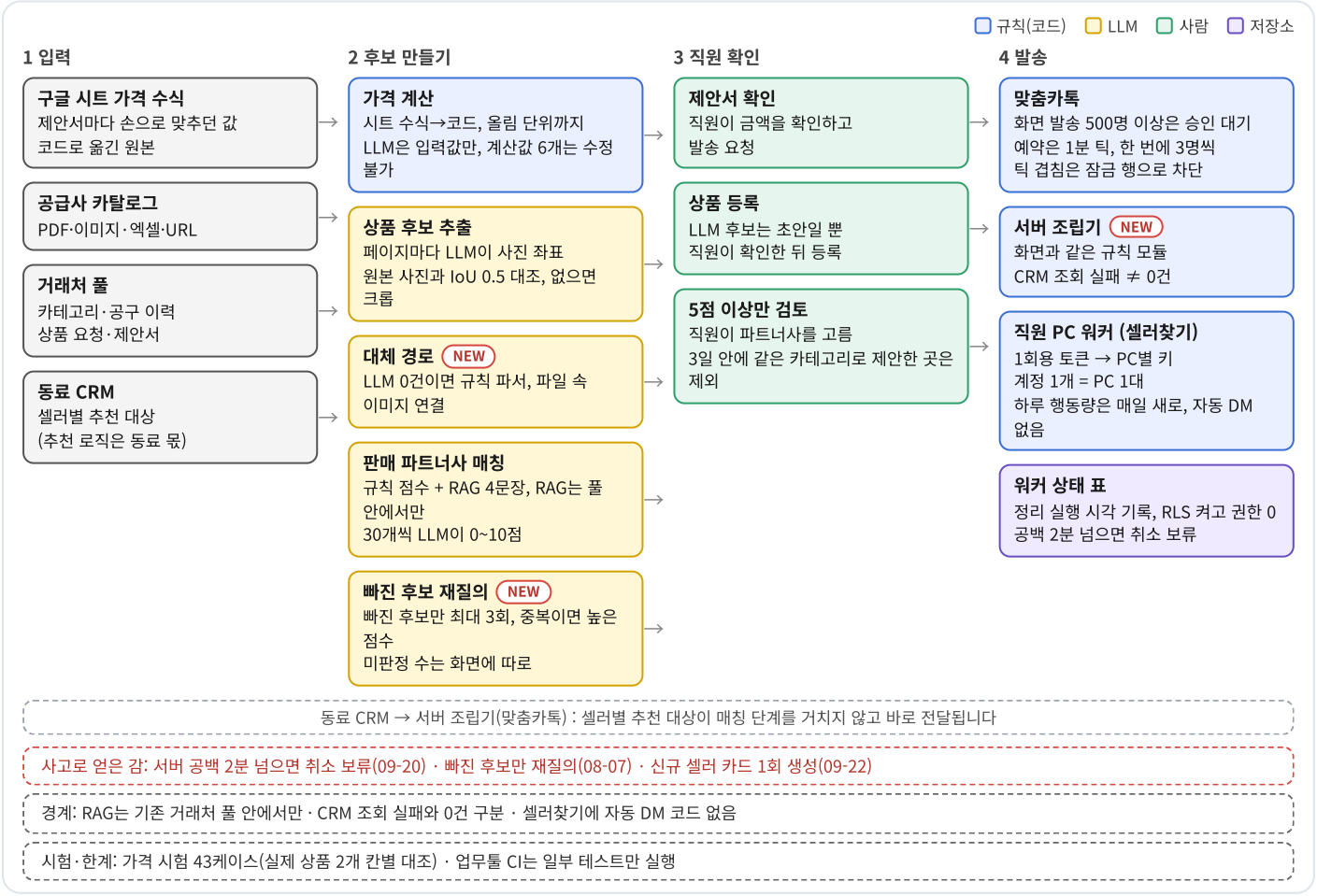
2. 규모 (항목별 기준일 표기)

- 맞춤카톡 하루 261명 발송 (2026-09-22)
- 가격 계산 시험 43케이스, 일부는 실제 상품의 시트 값과 칸별 대조
- 셀러찾기 워커 직원 PC 9대 (2026-09-18)

3. 어떻게 동작하나

① 가격: 시트 수식을 올림 단위까지 코드로 옮겼습니다. 수정 요청이 와도 LLM은 고칠 입력값 JSON만 내고, 금액은 코드가 다시 계산합니다.
② 카탈로그: PDF·이미지·엑셀·URL에서 LLM이 상품 후보를 뽑습니다. 직원이 확인한 뒤에만 등록됩니다.
③ 매칭: 규칙 점수 후보와 RAG 후보를 합쳐 30개씩 LLM이 0~10점을 매깁니다. 5점 이상만 직원이 보고, 3일 안에 같은 카테고리로 제안한 곳은 뺍니다.
④ 맞춤카톡: 실무자가 화면에서 발송·예약·멈추기를 합니다. 500명 이상은 승인 대기로 가고, 예약분은 서버가 1분마다 조금씩 보냅니다.
⑤ 셀러찾기: 직원 PC 워커가 계정당 PC 1대, 하루 행동 상한 안에서 돌아갑니다. 자동 DM 코드는 넣지 않았습니다.

Architecture



시트 수식→코드 계산 · 카탈로그·거래처 풀→LLM 후보→직원 확인 · CRM 대상→서버 조립→발송·승인 대기

사내 업무툴 · 가격·매칭·맞춤카톡

사고와 수정

1. 매칭 후보 누락(2026-07-15). 청크(나눠 보내는 묶음) 6개 중 3개에서 LLM이 후보를 다 판정하기 전에 응답을 끝냈습니다. 07-16부터 후보를 합집합으로 모은 뒤 나눠 묻고, 08-07부터는 빠진 후보만 최대 3회 다시 묻습니다. 판정 못 한 수는 화면에 따로 보입니다.

2. 워커 공유 키(2026-09-10). 직원 PC 워커 전부가 키 하나를 같이 쓰고 있었습니다. 30분짜리 1회용 설치 토큰으로 PC별 키를 받게 바꿨습니다. 서버에는 키의 sha256 값만 두고, 비교는 timingSafeEqual로 합니다.

3. 회차 6개 일괄 취소(2026-09-20). 서버가 이틀 멈췄다 돌아오자 '3분간 안 보인 워커' 규칙이 서버 공백을 워커 이탈로 읽어 진행 중 회차 6개를 한꺼번에 취소했습니다. 정리 실행 간격이 2분을 넘으면 서버 공백으로 보고 기준 시각을 되돌립니다.

기술 하이라이트

1. LLM이 고칠 수 있는 칸을 정했습니다. "셀러 수익금 1000으로" 같은 요청을 LLM이 역산하면 다른 금액까지 흔들립니다. 그래서 공급가·수익금처럼 계산되는 값 6개는 수정 목록에서 막았습니다. 이런 요청에는 바꿔야 할 입력값만 안내합니다. 마진율만 바꾸라고 하면 판매가는 돌려주지 않습니다.

2. 시트 수식을 올림 단위까지 옮겼습니다. 파트너 공급가는 단위 올림, 수수료율은 코드로 고정했습니다. 시트의 실제 상품 2개 (예시상품 30,000원 등)를 칸마다 대조하는 시험을 붙였습니다. 배분 비율을 바꿔도 총이익은 그대로이고, 두 몫의 합은 늘 총이익과 같다는 성질도 검사합니다.

3. PDF 안의 원본 사진을 먼저 찾습니다. 페이지를 한 장씩 LLM에 보내 사진 좌표를 받습니다. 그 좌표가 PDF에 든 원본 이미지와 IoU(두 상자가 겹친 넓이 비율) 0.5 넘게 겹치면 원본을 씁니다. 화면 80%를 넘는 배경은 후보에서 빼고, 맞는 원본이 없을 때만 페이지를 잘라 씁니다. 여러 페이지에 나온 상품은 빈 가격 칸을 서로 채웁니다.

4. 엑셀은 LLM이 실패해도 결과가 나옵니다. LLM 파싱이 0건이면 빈 화면 대신 규칙 파서가 같은 파일을 다시 읽습니다. 파일 안에 든 이미지는 순서대로 상품에 붙입니다. 750바이트 미만인 아이콘은 상품 사진으로 치지 않고 버립니다.

5. RAG는 후보를 더할 뿐 새로 만들지 못합니다. 규칙 점수는 카테고리, 공구 이력, 상품 요청, 제안서 순으로 매기고, RAG 4문장 질의 결과를 합집합으로 더합니다. RAG가 돌려준 ID는 기존 거래처 풀 안에 있을 때만 받습니다. n8n에 있던 이 로직은 vitest로 시험하려고 코드로 옮겼습니다.

6. 자동 발송과 화면 발송이 같은 규칙을 씁니다. 서버 조립기가 화면과 같은 상품 선택·템플릿 모듈을 불러, 같은 사람에게 다른 상품이 가지 않습니다. CRM 조회 실패와 0건은 따로 보고 0건일 때만 인기 상품으로 채웁니다. 신규 셀러 카드는 한 번만 만듭니다.

7. 예약 발송은 1분마다 조금씩, 겹치지 않게 보냅니다. 맞춤 추천이 붙는 대상은 한 틱에 3명, 추천 없는 대상은 30명까지 조립합니다. 안 나간 대기분이 5건 이상이면 그 틱은 건너뜁니다. 발송 시간창이 지나면 새로 시작하지 않아 밤에 몰아 보내지 않습니다. 틱이 겹치지 않게 예약마다 잠금 행을 두고, 잡을 때 남긴 토큰이 맞아야 풀 수 있게 했습니다.

8. 계정 하나는 PC 한 대에서만 돌아갑니다. 설치 토큰 확인과 사용 처리를 조건부 UPDATE 한 번으로 묶어, 한 토큰으로 두 PC가 동시에 등록하지 못합니다. 같은 인스타 계정이 두 PC에 걸리면 새로 로그인한 쪽이 이깁니다. 서버의 작업 배정 함수도 같은 규칙을 따릅니다. 하루 행동량은 매일 아침 범위 안에서 새로 뽑습니다.

내가 정한 것

- 금액은 LLM에 맡기지 않았습니다. 시트에서 옮긴 코드가 계산하고, LLM은 고칠 입력값만 돌려줍니다.
- RAG 검색은 후보를 넓히는 데만 씁니다. 기존 거래처 풀 밖의 업체는 들어오지 못합니다.
- 500명 이상 발송은 바로 보내지 않고 승인 대기에 둡니다.

PC 배포

직원 PC 배포 · 운영

StyleSeller · 사내 비공개 저장소

카톡·인스타 작업을 직원 PC 11대에 한 줄 설치로 배포하고 운영합니다 (2026-09-16 등록 기준)

기간	2026.02~현재(Instagram DM 확장 02, 카톡 도구 05~)
역할	설계: 설치·인증·갱신 구조, 서버 기준일(이전 메시지는 버리는 날짜) / 구현: 설치기, 워커, 윈도우 수집기, kakaocli 포크 수정 / 운영: 배포, 사고 대응, 감시 크론. AI 코딩 도구를 썼고, 범위는 제가 정하고 결과는 직접 확인했습니다
저장소	사내 저장소, kakaocli 포크

배경

카톡과 인스타에는 서버에서 부를 공식 API가 없어, 작업은 직원 맥과 윈도우 PC에서 돌아야 했습니다. 설치는 한 줄 명령, 인증은 기기별 키, 갱신은 도구별 자동 주기로 맞췄습니다. 서버도 직접 굴립니다. 클라우드 DB 한도로 업무툴 API가 모두 멈춘 2026-08-20에는 같은 API를 VM에 세워 그날 복구했습니다.

구조와 규모

1. 기술

macOS bash·Windows PowerShell 설치기, launchd와 작업 스케줄러, Node 워커, Python 수집기(윈도우), kakaocli 포크(Swift), VM Postgres+PostgREST+nginx, SSE(서버가 밀어 주는 이벤트) 푸시 허브, Docker 배포 스크립트, Slack 감시 크론

2. 규모 (항목별 기준일 표기)

- 직원 PC 11대(2026-09-16 등록 기준), 셀러찾기 워커 PC 9대(09-18)
- DB 4개 자체 호스팅 이전 (2026-08-06~08-25)
- 업무툴 API 전체 정지 당일 복구 (2026-08-20)

3. 어떻게 동작하나

① 직원은 한 줄 명령으로 설치합니다. 맥은 bash, 윈도우는 PowerShell입니다.
② 1회용 설치 토큰으로 등록하면 기기별 키를 받습니다. 서버에는 토큰과 키의 sha256만 남습니다.
③ 도구마다 자동 갱신 주기가 다르고, 갱신 시각은 PC마다 흩어 둡니다. 윈도우 수집기만 재설치로 바꿉니다.
④ 큐 작업은 지정된 기기만 집습니다. 집을 때는 조건을 건 UPDATE 한 번이라 두 기기가 같은 일을 잡지 못합니다.
⑤ 서버는 커밋한 이미지로만 배포합니다. 오픈채팅 미발송은 17:40 감시가 Slack으로 알리고, 사람이 확인한 뒤 다시 보냅니다.

Architecture



직원 맥·윈도우(한 줄 설치, 기기별 키, 흩어 둔 자동 갱신) → VM(PostgREST+nginx, 기준일, 커밋 이미지 배포) → 감시 크론과 Slack

직원 PC 배포 · 운영

사고와 수정

1. 업무툴 API 전체 정지(2026-08-20). 클라우드 DB 전송량 한도에 걸려 업무툴 API가 모두 멈추자 VM의 nginx가 Supabase 경로와 apikey 헤더 검증을 그대로 흉내 내게 했습니다. 앱은 접속 설정과 어댑터 몇 개만 바꿨고, 앱 코드의 createClient 호출 100여 곳은 그대로 둔 채 그날 복구했습니다.

2. 재설치가 멀쩡한 PC를 망가뜨림(2026-09-14). 설치기가 옛 설치를 먼저 지우고 나서 등록했습니다. 만료된 코드로 실행되자 설정도 패키지도 없는 PC가 남았습니다. 지금은 Node 확인, 등록, zip 해시 검증을 모두 통과해야 옛 폴더를 지웁니다. 기기 번호는 재설치해도 그대로 씁니다.

3. 옛 판 PC가 과거 대화를 통째로 올림(2026-09-16). 윈도우 3대가 쌓인 대화를 한꺼번에 보냈습니다. 시트 API가 429로 막혀 뒤에 설치한 2명은 행이 한 시간 넘게 비어 있었습니다. 클라이언트를 고쳐도 PC가 새 판을 받기 전에는 소용이 없으니 서버에 기준일을 두고, 그 이전 메시지는 200으로 받은 뒤 버립니다.

기술 하이라이트

1. 설치 토큰은 UPDATE 한 번으로 소진합니다. 조회와 사용 처리를 나누면 두 PC가 한 토큰으로 동시에 등록할 수 있습니다. used_at IS NULL과 만료 조건을 건 UPDATE가 행을 돌려줄 때만 통과시킵니다. 실패 이유는 직원이 읽을 문장으로 나눠 줍니다. 기기 한 대가 털리면 그 행만 폐기합니다.

2. 버전 번호 대신 tip 해시를 씁니다. 공개된 zip의 sha256이 곧 버전입니다. 버전 대장이 따로 없으니 대장과 파일이 어긋날 일도 없습니다. 워커는 잡 사이에만 교체하고 그냥 종료하며 재실행은 launchd와 작업 스케줄러 몫입니다. 스스로 다시 띄우다 둘이 뜨면 인스타 프로필이 깨집니다.

3. 갱신 시각은 PC마다 다르게 둡니다. 전 기기가 같은 분에 갱신하면 잘못된 배포를 거둘 틈이 없습니다. 맥 실시간 동기화는 14시에 분만 무작위로 고르고, 재설치해도 그 분을 지킵니다. 윈도우 CS 도구는 09:00~18:59 사이 무작위 시각에 받고, 로그온 2분 뒤 한 번 더 시도합니다.

4. 윈도우 카톡은 읽기 전용 수집기를 씁니다. 수집기는 읽기 권한만 열고 쓰기는 하지 않습니다.

5. 발송 성공은 exit code 말고 DB로 확인합니다. 2026-09-07에 사진만 가고 글이 빠졌는데 sent로 남았습니다. 이제 보낸 직후 카톡 로컬 DB를 읽고, 빠진 쪽만 그 자리에서 한 번 보충합니다. 사진이 안 보이면 업로드가 느린 경우를 감안해 더 기다립니다. 70명 방에 사진이 두 장 가는 건 유실보다 나쁩니다.

6. 기기 지정과 조건부 claim을 겹쳐 뒀습니다. 2026-07-29에 개발 맥이 원격 전용 맥과 같은 큐를 집었습니다. 성공과 실패가 번갈아 나와 원인을 찾는 데 하루가 걸렸습니다. 지금은 지정 호스트가 아니면 바로 끝납니다. claim은 status='pending' 조건부 UPDATE이고, 오픈채팅 큐는 UNIQUE(send_date, room_name)이 한 번 더 막습니다.

7. 기기 차단은 푸시로 바로 전달합니다. 푸시 허브는 Postgres LISTEN/NOTIFY를 받아 SSE로 워커를 깨웁니다. device_keys가 바뀌면 깨우기 전에 다시 인증합니다. 차단된 기기에는 revoked를 보내고 연결을 끊으니, 서버 쪽에서는 revoked_at 한 칸만 채우면 됩니다.

8. deploy.sh는 배포할 커밋의 판으로 재실행합니다. 2026-09-28 배포 실패는 도는 스크립트와 커밋의 스크립트가 달라서였습니다. 이미지 태그는 sha-<커밋>으로 한 번 정하고, pull이 실패하면 :latest로 물러나지 않고 멈춥니다. compose는 빌드 전에 렌더해 빠진 변수를 40분 빌드보다 먼저 잡습니다.

상태 · 판단 · 한계

- 운영 중입니다. 직원 PC 11대와 셀러찾기 워커 9대는 2026-09-16·09-18 등록 시점 수치이고, 그 뒤 대수는 여기 적지 않았습니다.
- 가장 잦은 사고는 두 기기가 한 자원을 쥐는 일이었습니다(07-29). 그래서 기기 지정을 기본으로 둡니다.
- 한계: 윈도우 수집기는 자동 갱신기가 없어 재설치로 바꿉니다. 수집 끊김 경보는 기본으로 꺼 두었습니다.
- 다시 한다면: 윈도우 수집기에도 처음부터 zip 해시 갱신기를 붙이겠습니다. 09-16 사고는 옛 판 PC가 클라이언트 수정을 받지 못해 커졌습니다.

내가 정한 것

- CI 이미지가 없으면 :latest로 대신하지 않고 배포를 멈추니, 배포된 이미지는 늘 커밋과 짝이 맞습니다.
- DB 장애 때 앱을 고치기보다 같은 API를 VM에 먼저 세운 덕에 앱 쪽 변경은 접속 설정과 어댑터 몇 개로 끝났습니다.
- 기준일 필터는 시각을 읽지 못한 메시지를 버리지 않습니다. 데이터가 조용히 사라지는 것보다 조금 더 들어오는 쪽이 눈에 보이고 고치기 쉽습니다.